Tutor Availability Platform

with Smart Recommendation and Waitlist Management

Milestone 2 Report

Course: Object Oriented Programming

Institute: IBA Karachi

Teacher: Samreen Kazi

Submitted By

Daniyyal Ahmed

Adarsh Sachdev

March 29, 2026

Contents

1 Introduction **3**

2 Project Objective **3**

3 Object Oriented Programming Concepts Used **3**

3.1 Inheritance 3

3.2 Polymorphism 4

3.3 Encapsulation 4

3.4 Abstraction 4

3.5 Exception Handling 4

4 System Overview **4**

5 Class Description **5**

5.1 User 5

5.2 Tutor 5

5.3 Student 6

5.4 TimeSlot 6

5.5 Waitlist 7

5.6 Session 7

5.7 RecommendationEngine 8

5.8 Platform 8

6 Smart Features Included **8**

6.1 Tutor Recommendation and Ranking 8

6.2 Waitlist Management 8

6.3 Automatic Slot Reassignment 9

6.4 Conflict Detection 9

7 Prototype Functionality Implemented **9**

8 UML Class Diagram **9**

9 Test Cases **10**

10 GitHub Repository Organization **11**

11 Conclusion **11**

1 Introduction

The Tutor Availability Platform is an Object Oriented Programming project developed to connect students with tutors in an efficient and structured manner. The system allows tutors to create profiles, define their subject expertise, manage available time slots, and handle session requests. Students can browse tutor profiles, search tutors by subject, request sessions, and keep track of their scheduled tutoring sessions.

This project goes beyond a basic management system by incorporating smart features such as tutor recommendation and ranking, waitlist management, and automatic slot reassignment after cancellations. These enhancements make the system more practical, intelligent, and user-oriented.

2 Project Objective

The main objective of this project is to design and implement a tutoring platform using proper Object Oriented Programming principles. The system aims to:

- manage tutors and students through a well-structured class hierarchy
- allow students to search tutors according to their academic needs
- support tutor availability and session booking
- prevent overlapping bookings through conflict detection
- manage waitlists when tutor slots are already occupied
- automatically assign cancelled slots to the next eligible student in the waitlist
- recommend and rank tutors based on subject match, rating, and availability

3 Object Oriented Programming Concepts Used

3.1 Inheritance

Inheritance allows a derived class to acquire the attributes and methods of a base (parent) class, promoting code reuse and a logical hierarchy. A base class User holds common attributes shared by all system users — userId, name, and contactInfo — along with their getters and setters. Both Tutor and Student inherit from User using public inheritance.

3.2 Polymorphism

Polymorphism means "many forms" — the same function name behaves differently depending on the object calling it. In C++, this is achieved through **virtual functions** and **function overriding**. **In this project:** The function viewProfile() is declared as a **pure virtual function** in the base class User. Both Tutor and Student override it with their own implementation, displaying different information depending on the object type.

3.3 Encapsulation

Encapsulation means bundling data (attributes) and the functions that operate on that data together inside a class, and restricting direct access to internal details using access specifiers (private, protected, public). All sensitive data is declared private inside every class. For example, in the Tutor class, attributes like subjects, rating, experienceYears, and availableSlots are private. No outside code can directly modify them. Instead, controlled access is provided through public getter and setter functions like getRating(), getName(), and setContactInfo().

3.4 Abstraction

Abstraction means hiding complex internal implementation details and exposing only what is necessary to the outside world. The user of a class does not need to know *how* it works internally, only *what* it does. The User class is made **abstract** by declaring viewProfile() as a pure virtual function (= 0). This means User cannot be instantiated directly — it serves only as a blueprint. This enforces a contract: any class that inherits from User *must* provide its own viewProfile() implementation.

3.5 Exception Handling

Exception handling in C++ uses try, catch, and throw to detect and respond to runtime errors gracefully, instead of letting the program crash. Exception handling is used to protect against invalid operations such as booking a session with a non-existent tutor, student, or slot ID. Instead of the program crashing silently, a descriptive error is thrown and caught.

4 System Overview

The system is based on the following main classes:

- User

- Tutor
- Student
- Session
- TimeSlot
- Waitlist
- RecommendationEngine
- Platform

The Platform class acts as the central controller. It stores the data of tutors, students, and sessions, and coordinates major system operations such as searching, booking, cancelling, and displaying results. The RecommendationEngine is responsible for intelligent tutor ranking.

5 Class Description

5.1 User

This is the parent class for all system users.

Attributes

- userId
- name
- contactInfo

Methods

- constructors
- getters and setters
- viewProfile()

5.2 Tutor

This class stores tutor-specific details.

Attributes

- subjects
- experienceYears
- rating
- availableSlots

Methods

- addSubject()
- addAvailability()
- viewProfile()
- isAvailable()

5.3 Student

This class stores student-specific information.

Attributes

- requestedSessions
- sessionHistory

Methods

- viewProfile()
- requestSession()

5.4 TimeSlot

This class represents a tutor's available time slot.

Attributes

- slotId
- date
- time
- isBooked

Methods

- displaySlot()
- bookSlot()
- freeSlot()

5.5 Waitlist

This class manages students waiting for a particular slot.

Attributes

- waitingStudents

Methods

- addStudent()
- getNextStudent()
- isEmpty()
- displayWaitlist()

5.6 Session

This class represents a tutoring session between a tutor and a student.

Attributes

- sessionId
- tutorName
- studentName
- subject
- slotInfo
- status

Methods

- displaySession()
- cancelSession()

5.7 RecommendationEngine

This class provides tutor recommendation and ranking.

Methods

- calculateScore()
- recommendTutors()

5.8 Platform

This is the main managing class of the system.

Methods

- addTutor()
- addStudent()
- showAllTutors()
- searchTutorsBySubject()
- recommendTutors()
- bookSession()
- cancelSession()
- showAllSessions()

6 Smart Features Included

6.1 Tutor Recommendation and Ranking

This class scores tutors and ranks them. calculateScore() gives points based on three things:

+50 if the tutor teaches the requested subject

+rating × 6 (so a 4.8 rated tutor gets +28.8 points)

+20 if the tutor has a free slot on the exact date and time requested

recommendTutors() runs this score for every tutor, collects the ones with score > 0, then sorts them highest to lowest using a simple bubble sort, and returns the sorted list of tutor pointers.

6.2 Waitlist Management

If a slot is already occupied, the student can be added to a waitlist for that slot.

6.3 Automatic Slot Reassignment

When a session is cancelled, the system checks the waitlist and automatically assigns the slot to the first waiting student.

6.4 Conflict Detection

The platform prevents double booking of the same tutor slot.

7 Prototype Functionality Implemented

The Milestone 2 prototype demonstrates the basic structure and core functionality of the platform. The implemented functionality includes:

- adding tutor profiles
- adding student profiles
- storing tutor subjects, ratings, and available slots
- viewing tutor and student profiles
- searching tutors by subject
- recommending tutors using ranking logic
- booking sessions
- handling full slots through waitlists
- cancelling sessions
- assigning a cancelled slot to the next student in the waitlist
- displaying all created sessions

The prototype is menu-driven and is designed to clearly show the core idea, structure, and intended workflow of the final project.

8 UML Class Diagram

The UML Class Diagram represents the architecture of the system and shows the classes, attributes, methods, and relationships used in the project.

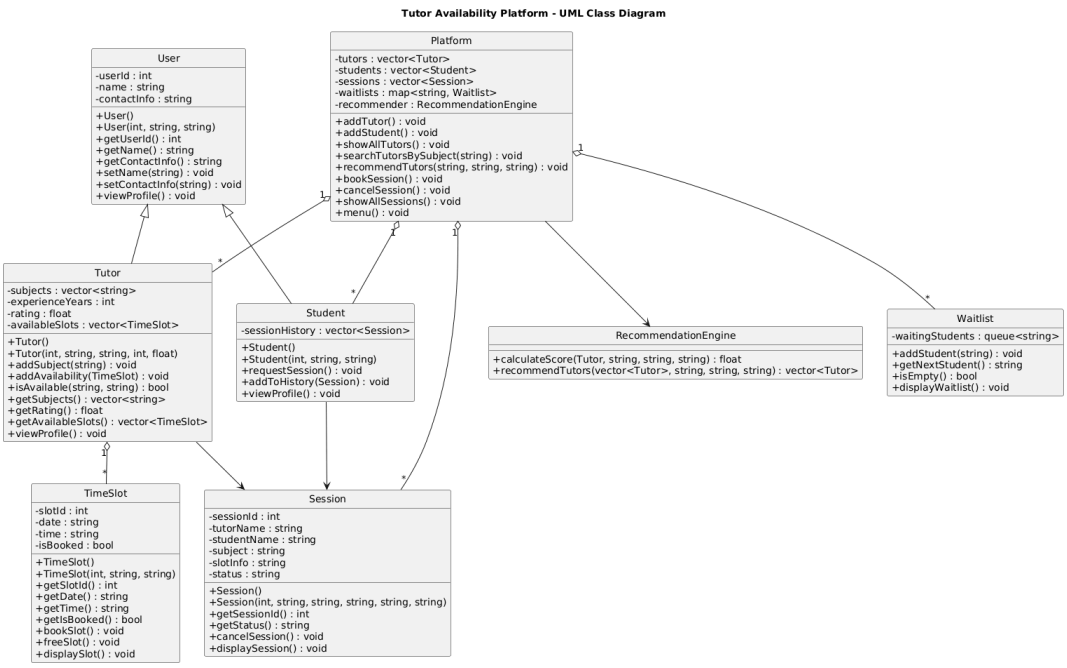


Figure 1: UML Class Diagram of Tutor Availability Platform

9 Test Cases

Test Case ID	Feature	Input	Output Confirms
TC-01	Add Tutor	Tutor name, contact, subject, rating, availability	3 tutor profiles created with subjects, rating, slots
TC-02	Add Student	Student name and contact details	3 student profiles created with contact info
TC-03	Search Tutors by Subject	Subject = Mathematics	Only Dr. Ali Raza and Ms. Sara Khan appear for ”Mathematics”
TC-04	Recommend Tutors	Subject = Physics, preferred slot = 4 PM	Mr. Usman Tariq ranks 1 for Physics at 4PM (subject match + available slot)
TC-05	Book Session	Student selects a tutor and an available slot	Session ID 1 created, Slot 1 changes from Available → Booked

TC-06	Conflict Detection	Another student requests the same booked slot	Zara and Hassan both get rejected and placed in the waitlist in order

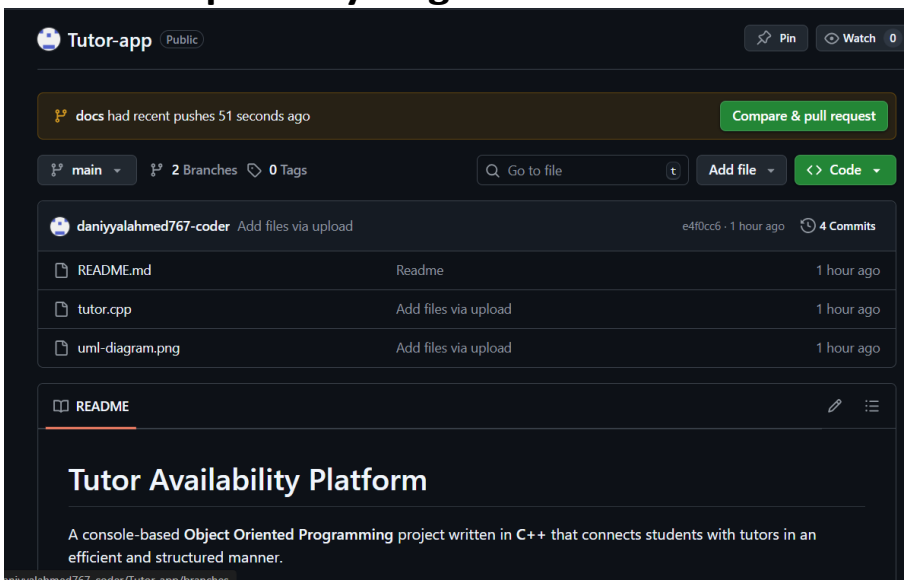
Sample Output / Screenshots

```
TC-05: Book Session
--------------------------------------------------
  Input: Ahmed Malik books Dr. Ali Raza for Mathematics, Slot 1
  Session booked successfully!
  +----------------------------------------+
  Session ID   : 1
  Tutor        : Dr. Ali Raza
  Student      : Ahmed Malik
  Subject      : Mathematics
  Slot         : 2025-06-10 10:00 AM
  Status       : Scheduled
  +----------------------------------------+

  [Result] Session created and Slot 1 is now marked as Booked.

  Verifying slot status in Dr. Ali Raza's profile:

  === Tutors teaching "Mathematics" ===

  ========== TUTOR PROFILE ==========
  ID          : 101
  Name        : Dr. Ali Raza
  Contact     : ali@email.com
  Experience  : 8 years
  Rating      : 4.8 / 5.0
  Subjects    : Mathematics, Physics
  Available Slots:
  Slot ID: 1 | Date: 2025-06-10 | Time: 10:00 AM | Status: Booked
  Slot ID: 2 | Date: 2025-06-11 | Time: 04:00 PM | Status: Available
  ==================================

  ========== TUTOR PROFILE ==========
  ID          : 102
  Name        : Ms. Sara Khan
  Contact     : sara@email.com
  Experience  : 5 years
  Rating      : 4.5 / 5.0
  Subjects    : Computer Science, Mathematics
  Available Slots:
  Slot ID: 3 | Date: 2025-06-10 | Time: 10:00 AM | Status: Available
```

```
--------------------------------------------------
TC-04: Recommend Tutors
--------------------------------------------------
  Input: Subject = "Physics", Date = "2025-06-13", Time = "04:00 PM"

  === Recommended Tutors for "Physics" on 2025-06-13 at 04:00 PM ===

  Rank #1 (Score: 95.2)
  ========== TUTOR PROFILE ==========
  ID          : 103
  Name        : Mr. Usman Tariq
  Contact     : usman@email.com
  Experience  : 6 years
  Rating      : 4.2 / 5.0
  Subjects    : Physics, Chemistry
  Available Slots:
  Slot ID: 5 | Date: 2025-06-13 | Time: 04:00 PM | Status: Available
  Slot ID: 6 | Date: 2025-06-14 | Time: 02:00 PM | Status: Available
  ==================================

  Rank #2 (Score: 78.8)
  ========== TUTOR PROFILE ==========
  ID          : 101
  Name        : Dr. Ali Raza
  Contact     : ali@email.com
  Experience  : 8 years
  Rating      : 4.8 / 5.0
  Subjects    : Mathematics, Physics
  Available Slots:
  Slot ID: 1 | Date: 2025-06-10 | Time: 10:00 AM | Status: Available
  Slot ID: 2 | Date: 2025-06-11 | Time: 04:00 PM | Status: Available
  ==================================

  Rank #3 (Score: 27)
  ========== TUTOR PROFILE ==========
  ID          : 102
  Name        : Ms. Sara Khan
  Contact     : sara@email.com
  Experience  : 5 years
  Rating      : 4.5 / 5.0
```

```
--------------------------------------------------
TC-06: Conflict Detection
--------------------------------------------------
  Slot 1 of Dr. Ali Raza is already booked by Ahmed Malik.
  Input: Zara Noor tries to book the same Slot 1.
  Slot is already booked. Adding "Zara Noor" to waitlist...
  [Waitlist] Student "Zara Noor" added to waitlist.

  Input: Hassan Butt also tries to book the same Slot 1.
  Slot is already booked. Adding "Hassan Butt" to waitlist...
  [Waitlist] Student "Hassan Butt" added to waitlist.

  [Result] Both students rejected from booking and added to waitlist.
  Displaying waitlist for Dr. Ali Raza, Slot 1:

  Waitlist for Tutor "Dr. Ali Raza", Slot 1:
  Waitlist:
    1. Zara Noor
    2. Hassan Butt


  Input: Hassan Butt also tries to book the same Slot 1.
  Slot is already booked. Adding "Hassan Butt" to waitlist...
  [Waitlist] Student "Hassan Butt" added to waitlist.

  [Result] Both students rejected from booking and added to waitlist.
  Displaying waitlist for Dr. Ali Raza, Slot 1:

  Waitlist for Tutor "Dr. Ali Raza", Slot 1:
  Waitlist:
    1. Zara Noor
    2. Hassan Butt
```

10 GitHub Repository Organization



Link: https://github.com/daniyyalahmed767-coder/Tutor-app

11 Conclusion

The Tutor Availability Platform is a practical Object Oriented Programming project that demonstrates core concepts such as inheritance, polymorphism, encapsulation and abstraction. In addition to profile and session management, it introduces smart decision-making through tutor recommendation and automated waitlist handling.

The Milestone 2 prototype establishes a strong design foundation for the final version of the project. It demonstrates not only the system structure and key class relationships, but also the main functional flows required for a scalable tutor scheduling platform.